

COMPOSED DATA TYPES

1. RECORD TYPE

- It represents a group of logically related data (for example, data about a customer: code, name, address are different in type but logically correlated). They can be manipulated as a unit;
- Each field (element of the structure) has a name and a data type;
- If we declare a variable of a record type, the fields are referred **to** **VARIABLE_NAME.FIELD_NAME**.

TYPE TYPE_NAME IS RECORD (field_name DATA_TYPE [,field_name DATA_TYPE:=|DEFAULT value]...);

VARIABLE_NAME TYPE_NAME;

Example:

Using a user-defined record data type, display the minimum price of the product with code 3133.

```
set serveroutput on
DECLARE
TYPE product_type IS RECORD (v_code product_information.product_id%type
NOT NULL:=3000,v_name product_information.product_name
%type,v_min_price product_information.min_price%type);
VREC_PROD product_type;
BEGIN
SELECT product_id, product_name, min_price into vrec_prod from
product_information where product_id=3133;
dbms_output.put_line('Product: '|| vrec_prod.v_name|| ' has the minimum price:
'||vrec_prod.v_min_price);
END;
/
```

To define a record-type variable based on the columns of a table, **%ROWTYPE** is used. In this case, the name of the items have the same name as the columns of the table, the same type of data, and are in the same order.

VARIABLE_NAME TABLE%ROWTYPE;

Examples:

Using a record data type of the same type as the products table displays the minimum price of the product with the code 3133.

```
DECLARE
vrec_prod products%rowtype;
BEGIN
SELECT * into vrec_prod from products where id_produs=3133;
dbms_output.put_line('Product: '|| vrec_prod.product_name|| ' has the minimum price:
'||vrec_prod.price_min);
```

```
END;  
/
```

Using a record data type of the same type as a row in the DEPARTMENTS table, display the name of each department with the id: 10, 20, 30, 40, 50.

```
DECLARE  
vrec_dep departments%rowtype;  
i number:=10;  
BEGIN  
loop  
SELECT * into vrec_dep from departments where department_id=i;  
dbms_output.put_line('Departament: '|| vrec_dep.department_id|| ' -  
'||vrec_dep.department_name);  
exit when i>=50; i:=i+10;  
end loop;  
END;  
/
```

2. Collection Data Types

- **Index-by** tables/associative arrays – are similar to scatter tables in other programming languages;
- **Nested-tables** – similar to sets in other programming languages;
- **Variable-length vectors** (VARRAYS) – similar to vectors in other programming languages.

Indexed arrays can only be used in PL/SQL blocks. Nested arrays as well as variable-length vectors can also be used at the schema level - they can be defined using CREATE TYPE SQL types and then columns in tables of that type.

Syntax for declaring indexed arrays:

```
TYPE TAB_NAME IS TABLE OF {DATA_TYPE[variable%type | table.column%type  
[NOT NULL]] table%rowtype}  
INDEX BY PLS_INTEGER|BINARY_INTEGER|VARCHAR2(dimension);
```

```
V_TAB TAB_NAME;
```

- The address is made with **v_tab(index).field**;
- DATA_TYPE can be any PL/SQL type except REF CURSOR;
- The index is unique, but in random order and can be negative. The range for PLS_INTEGER is (-2147483647, 2147483647);
- The following properties and methods can be used:

```
v_tab.EXISTS(i)  
v_tab.COUNT  
v_tab.FIRST și v_tab.LAST  
v_tab.PRIOR(i) și v_tab.NEXT(i)
```

v_tab.DELETE sau v_tab.DELETE(i) sau v_tab.DELETE(i,j)

Example:

--Using an indexed table of the type PRODUCTS. PRODUCT

```
DECLARE
--type declaration
type num_table is table of produse.denumire_produș%type index by pls_integer;
-- array variable declaration
v_tab num_table;
i number(5):=2252;
BEGIN
--loading in the array:
loop
SELECT denumire_produș into v_tab(i) from produse where id_produș=i;
i:=i+1;
exit when i>2255;
end loop;
--extraction from the array:
for i in v_tab.first.. v_tab.last loop
IF v_tab EXISTS(i) then
dbms_output.put_line('Product name: '|| v_tab(i));
end if;
end loop;
dbms_output.put_line('Total products in the indexed array: '|| v_tab.count);
END;
/
```

Example:

--Using an indexed array of the same type as a row in the EMPLOYEES table - %ROWTYPE

```
DECLARE
--declaring the type and variable
type emp_table is table of employees%rowtype index by pls_integer;
v_tab emp_table;
BEGIN
--loading in the array:
for i in 130..135 loop
SELECT * into v_tab(i) from employees where employee_id=i;
end loop;
--extraction from the array:
for i in v_tab.first.. v_tab.last loop
```

```
dbms_output.put_line('Employee '|| v_tab(i).first_name || ' works in the department:  
'||v_tab(i).department_id);  
end loop;  
dbms_output.put_line('Total employees in the indexed array: '|| v_tab.count);  
END;  
/
```